

فاز ۴: جاوا پیشرفته (Advanced Java) –

برنامه‌نویسی تابعی و همروندی

تسلط به این مباحث نشون میده که با روندهای جدید جاوا همراهی و می‌تونی کد تمیز، خواناتر و بهینه‌تری بنویسی.

این فاز شامل دو بخش بزرگه: برنامه‌نویسی تابعی (Functional Programming) با Lambda و Stream، و برنامه‌نویسی همروند (Concurrent Programming) با Thread ها. هر دوتاش توی مصاحبه و توی کار واقعی خیلی مهمن.

4.1 برنامه‌نویسی تابعی (Functional Programming) در جاوا

از Java 8 به بعد، جاوا دیگه فقط یه زبان شی‌گرا نیست. مفاهیم برنامه‌نویسی تابعی بهش اضافه شدن. هدف: نوشتن کدی که خواناتر، کوتاه‌تر و کم‌خطاتر باشه و راحت‌تر بتونیم پردازش‌های موازی انجام بدیم.

4.1.1 لامبدا (Lambda Expressions)

تعریف ساده: لامبدا یه تابع بی‌نام (Anonymous Function) هست. یعنی یه متد بدون اسم که می‌تونیم مثل یه شیء باهاش رفتار کنیم (به عنوان آرگومان پاسش بدیم، توی متغیر ذخیرش کنیم).

چرا لامبدا اومد؟ قبل از Java 8، برای تعریف یه رفتار ساده (مثلاً یه Thread جدید) مجبور بودیم یه کلاس بی‌نام (Anonymous Inner Class) بنویسیم که پر از کدهای اضافی و زشت بود.

```

1 // a. روش قدیمی با Anonymous Inner Class
2 Runnable oldWay = new Runnable() {
3     @Override
4     public void run() {
5         System.out.println("Running old way");
6     }
7 };
8
9 // روش جدید با Lambda (Java 8+)
10 Runnable newWay = () -> System.out.println("Running lambda way");

```

ساختار لامبدا

```
1 (param1, param2) -> { body }
```

• (پارامترها): ورودی‌های تابع. اگر فقط یک پارامتر باشد و نوعش رو ننویسی، می‌تونی پرانتز رو حذف کنی.

• ->: علامت فلش.

• { بدنه }: کد اجرایی. اگر فقط یک خط باشد، می‌تونی {} و return رو حذف کنی.

```

1 // a. مثال‌های معتبر لامبدا
2 () -> System.out.println("Hello"); // بدون پارامتر
3 x -> x * 2; // یک پارامتر، یک خط
4 (x, y) -> { // چند پارامتر، چند خط
5     int sum = x + y;
6     return sum;
7 };

```

! سوال مصاحبه‌ای ۱: Functional Interface (واسط تابعی) چیه و چه ربطی به لامبدا داره؟

داره؟

جواب: به اینترفیس که فقط و فقط یک متد انتزاعی داشته باشه. به این متد میگن Abstract Method. لامبدا دقیقاً پیاده‌سازی همین یک متد رو فراهم می‌کنه.

• کامپایلر می‌فهمه که لامبدا باید اون یک متد رو پیاده‌سازی کنه.

• برای جلوگیری از خطا، بهتره از Annotation مخصوص @FunctionalInterface استفاده کنیم. این Annotation کامپایلر رو مجبور می‌کنه چک کنه که اینترفیس واقعاً فقط یه متد انتزاعی داره.

```

1 @FunctionalInterface
2 interface Calculator {
3     int calculate(int x, int y); // فقط یک متد انتزاعی
4 }
5
6 public class Main {
7     public static void main(String[] args) {
8         // calculate استفاده از لامبدا برای پیاده‌سازی متد
9         Calculator adder = (a, b) -> a + b;
10        Calculator multiplier = (a, b) -> a * b;
11
12        System.out.println(adder.calculate(5, 3)); // خروجی: 8
13    }
14 }
```

بسته java.util.function: جاوا به سری Functional Interface آماده داره كه توی Stream API خیلی استفاده میشن. مهم‌ترین‌هاش

• `Predicate<T>`: به T میگیره، boolean برمیگردونه. (مثل filter كه تست شرطی انجام میده).

• `Function<T, R>`: به T میگیره، به R برمیگردونه. (مثل map كه تبدیل انجام میده).

• `Consumer<T>`: به T میگیره و هیچی برنمیگردونه (void). (مثل forEach).

• `Supplier<T>`: هیچی نمیگیره، به T جدید میسازه و برمیگردونه. (مثل `Collectors.toList()`).

Stream API 4.1.2

تعریف ساده: Stream به جریان (رودخونه) از داده‌هاست كه به صورت اعلامی (Declarative) روشن عملیات انجام میدیم. به جای اینکه بگیم چطور (با حلقه for)، می‌گیم چه کاری (filter, map, collect).

! سوال مصاحبه‌ای ۲: Stream با Collection چه فرقی داره؟

جواب حیاتی:

• ذخیره‌سازی: Collection به ساختمان داده است كه داده رو ذخیره می‌كنه. Stream اصلاً داده ذخیره نمی‌كنه،

فقط به مسیر برای انتقال و پردازش داده از به منبع (مثل Collection) هست.

• عملیات: Stream عملیات رو به دو دسته تقسیم می‌كنه: Terminal و Intermediate.

• Lazy Evaluation: به عملیات Intermediate (مثل filter) تا زمانی كه به عملیات Terminal (مثل collect) صدا

زده نشه، اصلاً اجرا نمی‌شه. این بهینه‌سازی خیلی مهمیه.

عملیات‌های Stream

ساخت Stream:

```
1 // a. از Collection
2 List<String> names = Arrays.asList("Ali", "Reza", "Sara");
3 Stream<String> stream1 = names.stream();
4
5 // از آرایه
6 Stream<String> stream2 = Arrays.stream(new String[]{"X", "Y"});
```

عملیات میانی (Intermediate): به Stream جدید برمیگردونن. Lazy هستند

• filter(Predicate): فیلتر بر اساس شرط.

• map(Function): تبدیل هر عنصر به یه چیز دیگه.

• sorted(): مرتب‌سازی.

• distinct(): حذف موارد تکراری.

• limit(n): فقط n عنصر اول.

عملیات پایانی (Terminal): به نتیجه (غیر Stream) برمیگردونن و جریان رو می‌بندن

• collect(Collectors.toList()): جمع‌آوری المان‌ها در یه List.

• forEach(Consumer): انجام عملی روی هر عنصر.

• reduce(BinaryOperator): ترکیب کل المان‌ها در یک مقدار واحد (مثلاً جمع کل).

4-Java-Advanced

```
1 // مثال کامل a.
2 List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
3
4 List<Integer> evenSquares = numbers.stream()
5     .filter(n -> n % 2 == 0)    // ۱. Intermediate: فقط زوج‌ها
6     .map(n -> n * n)           // ۲. Intermediate: به توان ۲ برسود
7     .sorted((a, b) -> b - a)   // ۳. Intermediate: نزولی مرتب کن
8     .limit(3)                  // ۴. Intermediate: فقط ۳ تا اول
9     .collect(Collectors.toList()); // ۵. Terminal: بریز توی یه لیست
10
11 // نتیجه: [36, 64, 100] (بزرگترین مربع‌های زوج: ۱۰، ۸ و ۶)
```

! سوال مصاحبه‌ای ۳: متد reduce چیکار می‌کنه؟

جواب: عملیات پایانی (Terminal) هست که عناصر Stream رو یکی یکی با هم طبق یه قاعده ترکیب می‌کنه تا به یه مقدار واحد برسیم. یه جورایی for حلقه‌ای هست که یه accumulator داریم.

```
1 // جمع اعداد به لیست a.
2 List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
3 int sum = numbers.stream()
4     .reduce(0, (subtotal, element) -> subtotal + element);
5 // 0 + 1 = 1
6 // 1 + 2 = 3
7 // 3 + 3 = 6
8 // 6 + 4 = 10 -> sum = 10
```

4.2 برنامه‌نویسی هم‌روند و چندریسمانی (Concurrency & Multithreading)

هدف: اجرای همزمان چندین بخش از برنامه برای استفاده بهتر از CPU و افزایش سرعت پاسخگویی. (به جای اینکه کاربر منتظر نمونه، به Thread دیگر کار رو انجام بده).

4.2.1 ساختن و اجرای Thread

دو روش اصلی:

1. `extends Thread`: کلاس خودت رو از کلاس `Thread` ارث ببر و متد `run()` رو `Override` کن. (روش قدیمی، محدودیت‌ها چون نمی‌تونی از کلاس دیگه‌ای ارث بگیری).

2. `implements Runnable` (توصیه شده): کلاس خودت `Runnable` رو پیاده‌سازی کنه، بعد به شیء `Thread` بسازی و `Runnable` رو بهش پاس بدی.

```

1 // روش توصیه شده با Runnable
2 class MyTask implements Runnable {
3     @Override
4     public void run() {
5         System.out.println("اجرا توسط: " + Thread.currentThread().getName());
6     }
7 }
8 public class Main {
9     public static void main(String[] args) {
10         Thread t1 = new Thread(new MyTask());
11         Thread t2 = new Thread(new MyTask());
12

```

```

13  t1.start(); // run() صدا نزن هیچوقت
14  t2.start(); // start() به Thread و جدید میسازه run() صدا میزنه
15  }
16  }

```

4.2.2 مشکلات رایج در همروندی

! سوال مصاحبه‌ای ۴: Race Condition (شرایط مسابقه) چیه؟

جواب: وقتی دو یا چند Thread به یه منبع مشترک (Shared Resource) دسترسی همزمان دارن و حداقل یکی از اون‌ها منبع رو تغییر میده. نتیجه نهایی به ترتیب زمان‌بندی Thread‌ها بستگی داره و غیرقابل پیش‌بینی و خطرناک میشه.

```

1 public class Counter {
2     private int count = 0;
3
4     // داره Race Condition! نیست Thread-Safe این متد
5     public void increment() {
6         count++; // این یه عملیات ۳ مرحله‌ایه: خواندن، افزودن، نوشتن
7     }
8 }

```

! سوال مصاحبه‌ای ۵: synchronized چطور مشکل Race Condition رو حل می‌کنه؟

جواب: کلمه کلیدی synchronized تضمین می‌کنه که در هر لحظه فقط یک Thread بتونه متد یا بلوک مشخصی از کد رو برای یک شیء خاص اجرا کنه. این کار با قفل کردن (Lock) انجام میشه.


```

1 public class SafeCounter {
2     private int count = 0;
3
4     public synchronized void increment() { // در هر لحظه Thread فقط یک
5         count++;
6     }
7
8     // روی به بلوک synchronized یا
9     public void incrementBlock() {
10        synchronized (this) {
11            count++;
12        }
13    }
14 }

```

نکته: synchronized باعث کاهش Performance میشه، چون Thread ها باید منتظر هم بمونن. باید فقط جایی که لازمه استفاده بشه

! سوال مصاحبه‌ای ۶: Deadlock (بن‌بست) چیه و چطور رخ میده؟

جواب: وضعیتی که دو یا چند Thread برای همیشه منتظر یکدیگر می‌مونن. معمولاً زمانی رخ میده که:

• Thread A قفل منبع ۱ رو داره و قفل منبع ۲ رو میخواد.

• Thread B قفل منبع ۲ رو داره و قفل منبع ۱ رو میخواد.

• هیچکدام قفل خودشون رو آزاد نمی‌کنن، پس هیچکدام به قفل بعدی نمی‌رسن. برنامه هنگ می‌کنه.

راه‌های جلوگیری:

1. ترتیب قفل‌گیری رو یکسان کن (همه اول قفل منبع ۱ رو بگیرن، بعد ۲).

2. از tryLock () با زمان مشخص استفاده کن تا اگر نتونست قفل کنه، قفل‌های قبلی رو آزاد کنه و دوباره تلاش کنه.

! سوال مصاحبه‌ای ۷: تفاوت Callable و Runnable؟

جواب:

• Runnable: متد run () داره که مقدار برمیگردونه و نمی‌تونه Checked Exception پرتاب کنه.

• Callable<V>: متد call () داره که یه مقدار از نوع V برمیگردونه و می‌تونه استثنا پرتاب کنه.

```
1 Callable<String> task = () -> {
```

```
2   Thread.sleep(1000);
```

```
3   return "نتیجه کار";
```

```
4};
```

```
5 // استفاده کرد ExecutorService باید از Callable برای اجرای
```

! سوال مصاحبه‌ای ۸: ExecutorService و Thread Pool چیه؟ چرا ازشون استفاده

می‌کنیم؟

جواب: ساختن و از بین بردن Thread ها هزینه‌ی زیادی داره. به جای ساختن یه Thread جدید برای هر کار کوچیک (که ممکنه هزاران کار همزمان باشه)، از یه استخر (Pool) از Thread های آماده استفاده می‌کنیم.

• ExecutorService یه اینترفیس جاوا برای مدیریت این استخرهاست.

• کارها رو به صورت Runnable یا Callable بهش میدیم (submit).

• خودش یه Thread آزاد از استخر برمی‌داره و کار رو اجرا می‌کنه.

• بعد از اتمام، Thread رو نمی‌کشه، برش میگردونه به استخر برای کار بعدی.

```
1 ExecutorService executor = Executors.newFixedThreadPool(5); // استخری با ۵ Thread
2
3 for (int i = 0; i < 10; i++) {
4     executor.submit(() -> {
5         System.out.println(Thread.currentThread().getName() + " is running");
6     });
7 }
8
9 executor.shutdown(); // بعد از اتمام کارها، استخر رو خاموش کن
```

این هم از فاز ۴. با Stream API و Lambda می‌تونن کد خیلی تمیزتری بنویسن، و با Concurrency می‌فهمی برنامه‌های بزرگ چطور سریع و پاسخگو باقی میمونن.

! سوال مصاحبه‌ای ۹: CompletableFuture چیه و چه کاربردی داره؟

جواب: CompletableFuture یه راه مدرن برای برنامه‌نویسی ناهمگام (Asynchronous) در جاواست. به ما اجازه میده:

• چندین Task رو به صورت زنجیره‌ای اجرا کنیم

• Taskها رو ترکیب کنیم

• خطاها رو مدیریت کنیم

```
1 // به صورت ناهمگام Task اجرای به a.
2 CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
3     // کار سنگین
4     return "Result";
5 });
```

```

6
7 // b. زنجیره‌ای کردن عملیات
8 CompletableFuture.supplyAsync(() -> "Hello")
9 .thenApply(String::toUpperCase) // HELLO
10 .thenApply(s -> s + " World") // HELLO World
11 .thenAccept(System.out::println); // چاپ نتیجه
1
2 // c. Task ترکیب دو
3 CompletableFuture<Integer> f1 = CompletableFuture.supplyAsync(() -> 5);
4 CompletableFuture<Integer> f2 = CompletableFuture.supplyAsync(() -> 3);
5 f1.thenCombine(f2, (a, b) -> a + b) // 8
6 .thenAccept(System.out::println);

```

! سوال مصاحبه‌ای ۱۰: Optional چیست و چرا استفاده می‌شود؟

جواب: Optional یک Container است که می‌تونه یه مقدار داشته باشه یا خالی باشه (null). هدفش جلوگیری از NullPointerException هست.

```

1 // a. روش قدیمی (خطرناک)
2 String name = getUsername();
3 if (name != null) {
4     System.out.println(name.length()); // NPE ممکنه
5 }
6
7 // b. Optional روش جدید با
8 Optional<String> optionalName = Optional.ofNullable(getUsername());
9 optionalName.ifPresent(name -> System.out.println(name.length()));
10

```

```

11 // Optional متدهای مفید c.
12 optionalName.orElse("Default"); // بود، مقدار پیش فرض null اگر
13 optionalName.orElseThrow(() -> new RuntimeException("Name not found"));
14 optionalName.map(String::toUpperCase).orElse("EMPTY");

```

! سوال مصاحبه‌ای ۱۱: متدهای Default و Static در Interface (از Java 8) چین؟

جواب: قبل از جاوا ۸، Interface ها فقط می‌توانستند متدهای abstract داشته باشند. از جاوا ۸:

Default Method: متدی با بدنه که می‌تونه در Interface باشه و کلاس‌های implement کننده می‌تونن override کنن یا نکنن.

Static Method: متدی که به خود Interface تعلق داره و با اسم Interface صدا زده میشه.

```

1  interface Vehicle {
2      // Abstract Method
3      void start();
4
5      // Default Method (بدنه داره)
6      default void stop() {
7          System.out.println("Vehicle stopped");
8      }
9
10     // Static Method
11     static void service() {
12         System.out.println("Vehicle in service");
13     }
14 }
15
16 class Car implements Vehicle {

```

4-Java-Advanced

```
17  @Override
18  public void start() {
22      System.out.println("Car started");
23  }
1  // stop() رو override نکرده، پس از Interface استفاده می‌کنه
2  }
3
4  // استفاده
5  Car car = new Car();
6  car.start(); // Car started
7  car.stop(); // Vehicle stopped
8  Vehicle.service(); // Vehicle in service
```